

Esplorazione Telemetria: Degradazione Motori Turbofan (NASA CMAPSS)

Riccardo Baratto

2026-08-25

Contents

Introduzione e Contesto dei Sensori	1
1. Caricamento e Pulizia dei Dati	2
2. Isolamento di un Singolo Motore	2
3. Visualizzazione della Telemetria	3
4. Feature Engineering: Calcolo del RUL (Remaining Useful Life)	4
5. Pulizia delle Feature a Varianza Quasi-Zero	4
6. Suddivisione Train / Validation per Motori	5
7. Addestramento del Modello (Random Forest)	5
8. Importanza delle Variabili (Feature Importance)	5
9. Valutazione Formale: RMSE e Score Asimmetrico NASA	6
10. Visualizzazione Grafica degli Errori (Previsioni vs Realtà)	7
11. Analisi dei Residui del Modello	8

Introduzione e Contesto dei Sensori

La manutenzione predittiva nel settore aerospaziale affronta la sfida critica di stimare il *Remaining Useful Life* (RUL) di componenti complessi prima che si verifichi un guasto strutturale. Per analizzare questo fenomeno, il presente studio utilizza il dataset **NASA C-MAPSS (Commercial Modular Aero-Propulsion System Simulation)**, focalizzandosi sul sotto-dataset FD001. Il set di dati raccoglie serie temporali multivariate generate da una flotta di motori turbofan identici. Ciascun motore parte in condizioni di perfetta efficienza operativa e sviluppa progressivamente una condizione di degrado (localizzata nel compressore ad alta pressione - HPC) fino al punto di rottura definitiva.

Il sistema registra complessivamente 26 colonne per ogni ciclo di volo, suddivise in tre categorie principali:

Identificativi e Tempo: L'ID univoco del motore (*EngineID*) e il numero di cicli di volo trascorsi (*Cycle*).

Parametri Operativi (*OpSettings*): Tre diverse impostazioni operative (*OpSetting1*, *OpSetting2*, *OpSetting3*) che definiscono le condizioni di volo (altitudine, numero di Mach e angolo della leva di spinta - TRA). Nel caso specifico del dataset FD001, i motori operano in un'unica condizione costante a livello del mare.

Misurazioni dei Sensori Fisici (*Sensor1* a *Sensor21*): Un set di 21 canali di telemetria che monitorano in tempo reale grandezze termodinamiche e cinematiche chiave del motore. Tra questi figurano temperature totali in vari punti strategici del flusso (es. *Sensor2*, *Sensor4*), pressioni statiche e totali, velocità fisiche e corrette degli alberi motore (*Nf*, *Nc*), e flussi di raffreddamento come il bleed della turbina di bassa pressione (*Sensor21*). Tali sensori catturano la risposta dinamica del sistema e l'evoluzione graduale del danno strutturale mascherata da rumore di processo e di misura.

1. Caricamento e Pulizia dei Dati

I dati grezzi sono privi di intestazione. Le prime due colonne indicano l'ID del motore e il ciclo di volo (il tempo). Le successive tre sono i settaggi operativi, mentre le ultime 21 sono le letture dei sensori fisici.

```
#train_FD001.txt
df_turbofan <- read_table("/home/riccardo/Scrivania/Anomaly detection/6. Turbofan Engine Degradation Sim

nomi_colonne <- c("EngineID", "Cycle", "OpSetting1", "OpSetting2", "OpSetting3", paste0("Sensor", 1:21))
df_turbofan <- df_turbofan[, 1:26]
colnames(df_turbofan) <- nomi_colonne
glimpse(df_turbofan)
```

```
## Rows: 20,631
## Columns: 26
## $ EngineID    <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
## $ Cycle       <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, ~
## $ OpSetting1  <dbl> -0.0007, 0.0019, -0.0043, 0.0007, -0.0019, -0.0043, 0.0010, ~
## $ OpSetting2  <dbl> -4e-04, -3e-04, 3e-04, 0e+00, -2e-04, -1e-04, 1e-04, 3e-04, ~
## $ OpSetting3  <dbl> 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, ~
## $ Sensor1     <dbl> 518.67, 518.67, 518.67, 518.67, 518.67, 518.67, 518.67, 518.~
## $ Sensor2     <dbl> 641.82, 642.15, 642.35, 642.35, 642.37, 642.10, 642.48, 642~
## $ Sensor3     <dbl> 1589.70, 1591.82, 1587.99, 1582.79, 1582.85, 1584.47, 1592.~
## $ Sensor4     <dbl> 1400.60, 1403.14, 1404.20, 1401.87, 1406.22, 1398.37, 1397.~
## $ Sensor5     <dbl> 14.62, 14.62, 14.62, 14.62, 14.62, 14.62, 14.62, 14.62, 14.~
## $ Sensor6     <dbl> 21.61, 21.61, 21.61, 21.61, 21.61, 21.61, 21.61, 21.61, 21.~
## $ Sensor7     <dbl> 554.36, 553.75, 554.26, 554.45, 554.00, 554.67, 554.34, 553~
## $ Sensor8     <dbl> 2388.06, 2388.04, 2388.08, 2388.11, 2388.06, 2388.02, 2388.~
## $ Sensor9     <dbl> 9046.19, 9044.07, 9052.94, 9049.48, 9055.15, 9049.68, 9059.~
## $ Sensor10    <dbl> 1.3, 1.3, 1.3, 1.3, 1.3, 1.3, 1.3, 1.3, 1.3, 1.3, 1.3, 1.3, ~
## $ Sensor11    <dbl> 47.47, 47.49, 47.27, 47.13, 47.28, 47.16, 47.36, 47.24, 47.~
## $ Sensor12    <dbl> 521.66, 522.28, 522.42, 522.86, 522.19, 521.68, 522.32, 522~
## $ Sensor13    <dbl> 2388.02, 2388.07, 2388.03, 2388.08, 2388.04, 2388.03, 2388.~
## $ Sensor14    <dbl> 8138.62, 8131.49, 8133.23, 8133.83, 8133.80, 8132.85, 8132.~
## $ Sensor15    <dbl> 8.4195, 8.4318, 8.4178, 8.3682, 8.4294, 8.4108, 8.3974, 8.4~
## $ Sensor16    <dbl> 0.03, 0.03, 0.03, 0.03, 0.03, 0.03, 0.03, 0.03, 0.03, 0.03, ~
## $ Sensor17    <dbl> 392, 392, 390, 392, 393, 391, 392, 391, 392, 393, 392, 391, ~
## $ Sensor18    <dbl> 2388, 2388, 2388, 2388, 2388, 2388, 2388, 2388, 2388, 2388, ~
## $ Sensor19    <dbl> 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, ~
## $ Sensor20    <dbl> 39.06, 39.00, 38.95, 38.88, 38.90, 38.98, 39.10, 38.97, 39.~
## $ Sensor21    <dbl> 23.4190, 23.4236, 23.3442, 23.3739, 23.4044, 23.3669, 23.37~
```

2. Isolamento di un Singolo Motore

Per capire come si comporta un motore prima di cedere, isoliamo la telemetria dell'Engine numero 1. Il suo ultimo "Cycle" registrato nel dataset di training rappresenta il momento del guasto (Run-to-Failure).

```
df_motore1 <- df_turbofan %>%
  filter(EngineID == 1)

#(RUL - Remaining Useful Life)
cicli_totali_motore1 <- max(df_motore1$Cycle)
print(paste("Il motore 1 si è guastato al ciclo:", cicli_totali_motore1))

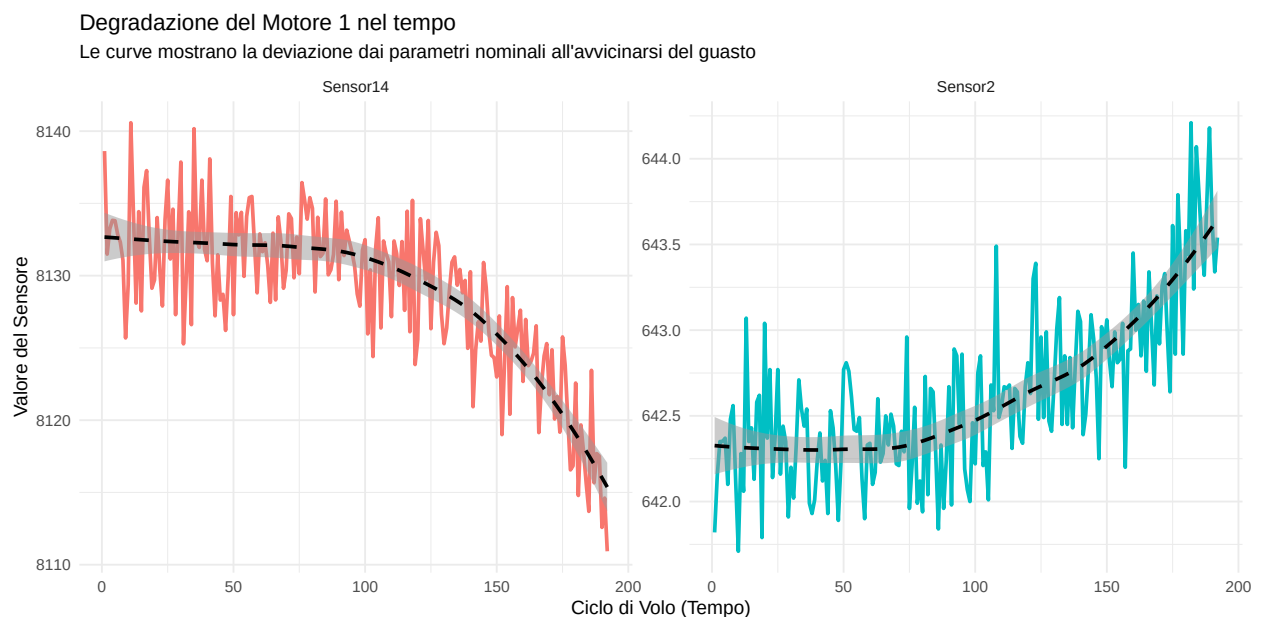
## [1] "Il motore 1 si è guastato al ciclo: 192"
```

3. Visualizzazione della Telemetria

Tracciamo il “Sensor 2” e il “Sensor 14” (Velocità del core) per visualizzare l’anomalia che cresce prima del cedimento.

```
df_plot <- df_motore1 %>%
  select(Cycle, Sensor2, Sensor14) %>%
  pivot_longer(cols = starts_with("Sensor"),
    names_to = "Sensore",
    values_to = "Valore")

ggplot(df_plot, aes(x = Cycle, y = Valore, color = Sensore)) +
  geom_line(linewidth = 1) +
  geom_smooth(method = "loess", linetype = "dashed", color = "black", alpha = 0.5) +
  facet_wrap(~ Sensore, scales = "free_y") +
  labs(
    title = "Degradazione del Motore 1 nel tempo",
    subtitle = "Le curve mostrano la deviazione dai parametri nominali all'avvicinarsi del guasto",
    x = "Ciclo di Volo (Tempo)",
    y = "Valore del Sensore"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```



Per il sensore 14 osserviamo che man mano si avvicina alla fine della vita del motore, il valore subisce un crollo drastico e marcato, mentre al contrario nel sensore 2 osserviamo che la temperatura/pressione cresce visibilmente verso l'altro man mano che il motore si usura e perde di efficienza.

4. Feature Engineering: Calcolo del RUL (Remaining Useful Life)

Nel dataset di training, ogni motore vola fino alla rottura definitiva. Il target da prevedere è il numero di cicli rimanenti prima del guasto per ciascun record. Per farlo, calcoliamo la differenza tra il ciclo massimo di vita di ciascun motore e il ciclo corrente.

Inoltre, applichiamo un limite superiore (*cap*) a 125 cicli, come ampiamente suggerito nella letteratura scientifica sul C-MAPSS, poiché nelle prime fasi di vita la stima a lungo termine è piatta e non lineare.

```
#ciclo massimo per ogni motore
df_rul <- df_turbofan %>%
  group_by(EngineID) %>%
  summarise(MaxCycle = max(Cycle))

#RUL
df_model <- df_turbofan %>%
  left_join(df_rul, by = "EngineID") %>%
  mutate(RUL = MaxCycle - Cycle) %>%
  select(-MaxCycle)

#cap a 125 cicli
df_model <- df_model %>%
  mutate(RUL = pmin(RUL, 125))

summary(df_model$RUL)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      0.00   51.00  103.00   86.83  125.00  125.00
```

5. Pulizia delle Feature a Varianza Quasi-Zero

Prima di passare l'intero dataset a un modello predittivo, eliminiamo le colonne che non variano (come i sensori o i parametri operativi che rimangono costanti in questa specifica condizione di test).

```
##pulizia delle Feature a Varianza Quasi-Zero

vector_rul <- df_model$RUL
df_features <- df_model %>% select(-RUL, -EngineID, -Cycle)
nzv_info <- nearZeroVar(df_features, saveMetrics = TRUE)
colonne_da_rimuovere <- rownames(nzv_info[nzv_info$nzv == TRUE, ])

print("Variabili predittive scartate per varianza quasi nulla:")

## [1] "Variabili predittive scartate per varianza quasi nulla:"

print(colonne_da_rimuovere)
```

```
## [1] "OpSetting3" "Sensor1"      "Sensor5"      "Sensor6"      "Sensor10"
## [6] "Sensor16"   "Sensor18"      "Sensor19"
```

```
df_clean <- df_features %>%
  select(-all_of(colonne_da_rimuovere))

df_clean$EngineID <- df_model$EngineID
df_clean$Cycle <- df_model$Cycle
df_clean$RUL <- vector_rul
```

6. Suddivisione Train / Validation per Motori

Per evitare data leakage (cioè che il modello veda informazioni relative a un motore sia in fase di training che di test), dividiamo i dati a livello di EngineID: l'80% dei motori andrà nel training set e il 20% nel validation set.

```
set.seed(123)
motori_unici <- unique(df_clean$EngineID)
train_engines <- sample(motori_unici, size = 0.8 * length(motori_unici))
train_data <- df_clean %>% filter(EngineID %in% train_engines) %>% select(-EngineID, -Cycle)
val_data <- df_clean %>% filter(!EngineID %in% train_engines) %>% select(-EngineID, -Cycle)
cat("Righe nel Training Set:", nrow(train_data), "\n")
```

```
## Righe nel Training Set: 16560
```

```
cat("Righe nel Validation Set:", nrow(val_data), "\n")
```

```
## Righe nel Validation Set: 4071
```

7. Addestramento del Modello (Random Forest)

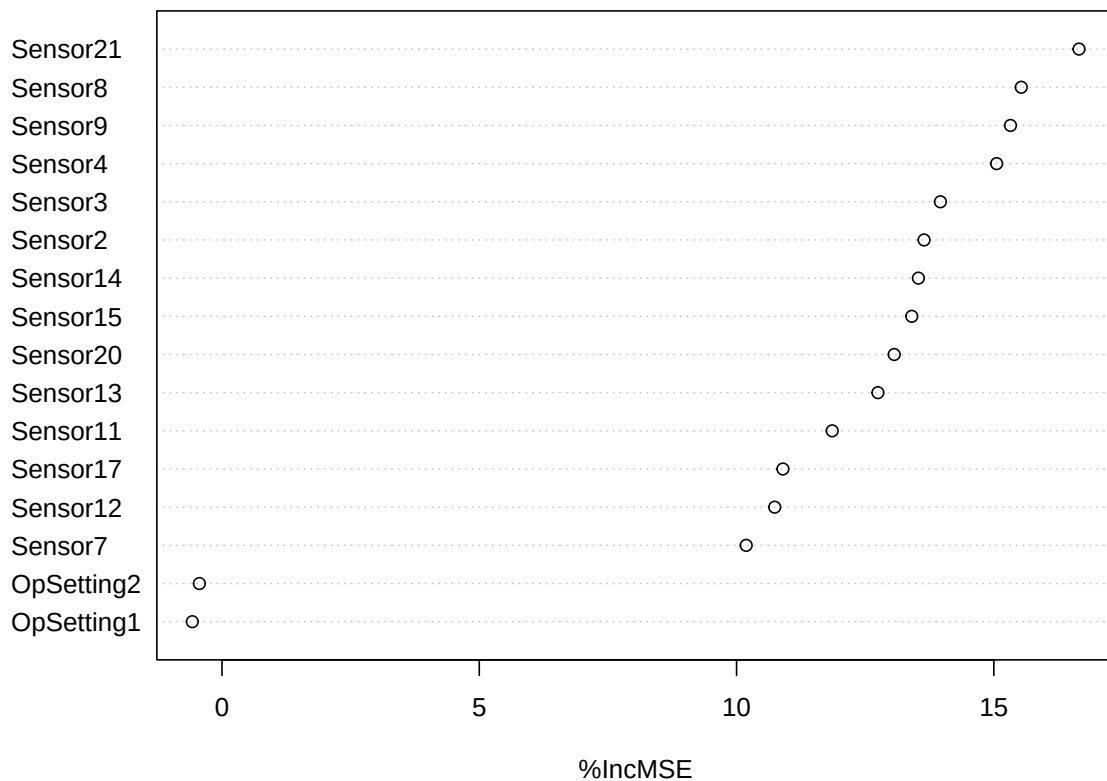
```
mod_rf <- randomForest(
  RUL ~ .,
  data = train_data,
  ntree = 40,
  importance = TRUE
)
```

8. Importanza delle Variabili (Feature Importance)

Vediamo quali sensori hanno pesato di più nel calcolo del Random Forest per stimare la vita residua del motore.

```
varImpPlot(mod_rf, type = 1, main = "Importanza dei Sensori nella Previsione del RUL")
```

Importanza dei Sensori nella Previsione del RUL



Dall'analisi dell'importanza delle variabili, si osserva come i sensori guida del degrado siano i sensori 21 (LPT coolant bleed), 8 (Physical fan speed), 9 (Physical core speed) e 4 (Total temperature at LPT outlet). Una loro alterazione provoca un incremento dell'errore predittivo compreso tra i 15 e i 17 punti percentuali, andando quindi a confermare come i canali predittivi per tracciare la progressione del cedimento del compressore ad alta pressione (HPC). Anche i sensori 3, 2, 14 e 15 mostrano un contributo solido e costante, catturando la deviazione termodinamica e di velocità via via che il motore perde efficienza.

9. Valutazione Formale: RMSE e Score Asimmetrico NASA

Calcoliamo le prestazioni del nostro Random Forest sul set di validazione utilizzando sia l'errore quadratico medio sia la funzione di costo ufficiale del concorso NASA PHM08.

```
pred_val <- predict(mod_rf, val_data)
val_rul <- val_data$RUL
rmse_val <- sqrt(mean((val_rul - pred_val)^2))
cat("RMSE sul Validation Set:", round(rmse_val, 2), "\n")
```

```
## RMSE sul Validation Set: 19.09
```

```
# funzione di Score Asimmetrico NASA PHM08
#d = pred_RUL - true_RUL (Errore di previsione)
#se d < 0 (predizione in anticipo): penalità esponenziale con a1 = 10
#se d >= 0 (predizione in ritardo): penalità esponenziale più severa con a2 = 13
```

```

nasa_score <- function(true_ruls, pred_ruls) {
  d <- pred_ruls - true_ruls
  score <- ifelse(d < 0,
                  exp(-d / 10) - 1,
                  exp(d / 13) - 1)
  return(sum(score))
}

score_totale <- nasa_score(val_rul, pred_val)
cat("Score Asimmetrico NASA Totale:", round(score_totale, 2), "\n")

```

```
## Score Asimmetrico NASA Totale: 25548.72
```

L'applicazione della funzione di score asimmetrica della nasa fornisce una metrica di costo orientata alla sicurezza industriale, evidenziando una buona simmetria nelle stime.

10. Visualizzazione Grafica degli Errori (Previsioni vs Realtà)

Un grafico di dispersione ci permette di vedere se il modello tende a sovrastimare o sottostimare il RUL.

```

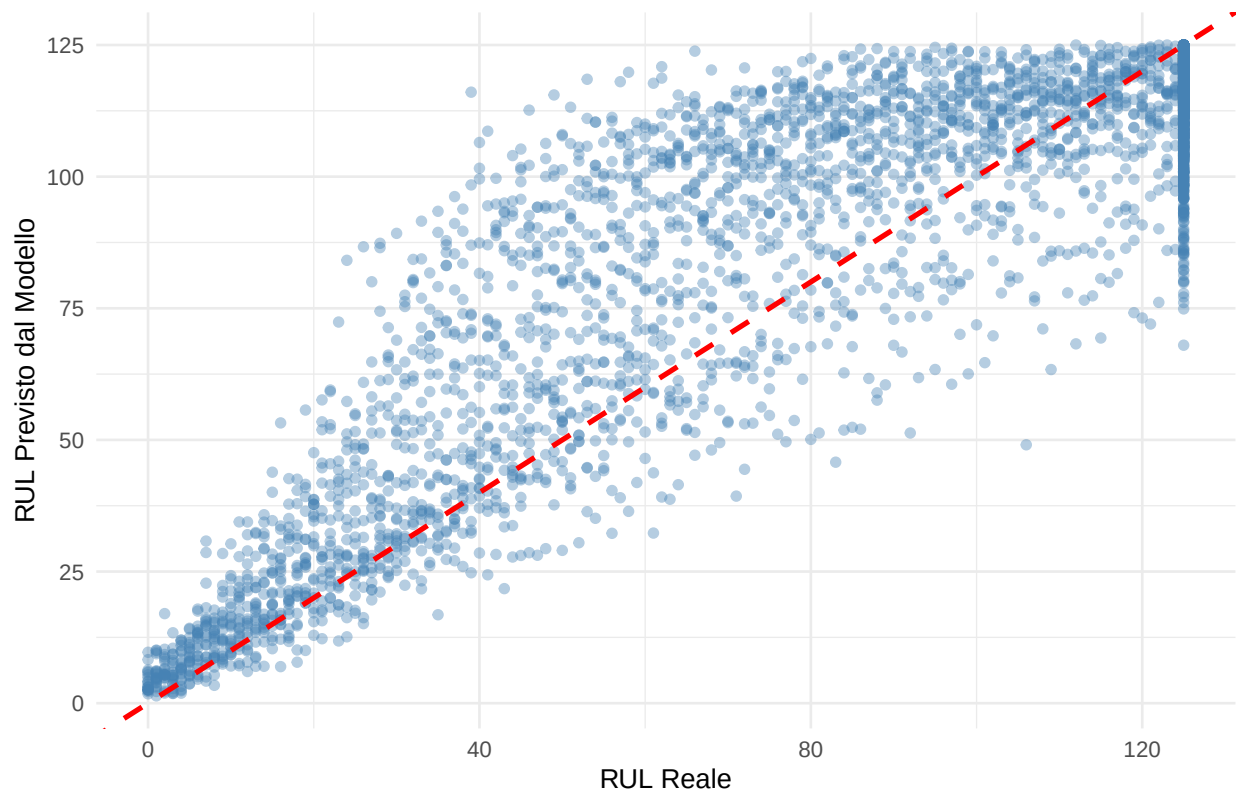
df_risultati <- data.frame(
  True_RUL = val_rul,
  Pred_RUL = pred_val
)

ggplot(df_risultati, aes(x = True_RUL, y = Pred_RUL)) +
  geom_point(alpha = 0.4, color = "steelblue") +
  geom_abline(intercept = 0, slope = 1, color = "red", linetype = "dashed", linewidth = 1) +
  labs(
    title = "Confronto RUL Reale vs RUL Previsto (Random Forest)",
    subtitle = "La linea rossa rappresenta la predizione perfetta (y = x)",
    x = "RUL Reale",
    y = "RUL Previsto dal Modello"
  ) +
  theme_minimal()

```

Confronto RUL Reale vs RUL Previsto (Random Forest)

La linea rossa rappresenta la predizione perfetta ($y = x$)



IL modello mostra una buona simmetria complessiva, ma nelle fase intermedie tende leggermente a rimanere sopra la linea rossa (sovrastima del RUL), questo si riflette anche nella funzione asimmetrica della Nasa.

11. Analisi dei Residui del Modello

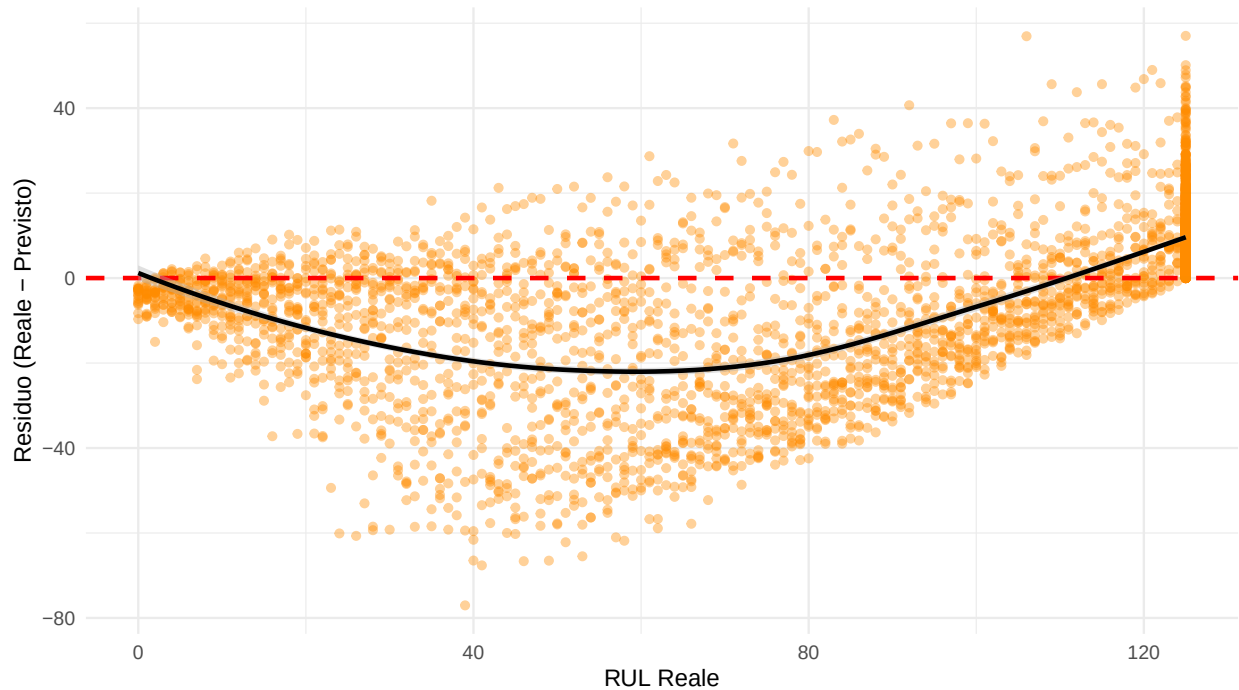
Per approfondire la diagnostica del modello, analizziamo i residui ($\text{Residuo} = \text{RUL Reale} - \text{RUL Previsto}$) in funzione del tempo di vita del motore. Questo ci permette di verificare la presenza di eventuali distorsioni sistematiche (bias) lungo le diverse fasi operative.

```
# Calcolo dei residui
df_risultati$Residuo <- df_risultati$True_RUL - df_risultati$Pred_RUL

# Grafico dei residui vs RUL Reale
ggplot(df_risultati, aes(x = True_RUL, y = Residuo)) +
  geom_point(alpha = 0.4, color = "darkorange") +
  geom_hline(yintercept = 0, color = "red", linetype = "dashed", linewidth = 1) +
  geom_smooth(method = "loess", color = "black", se = TRUE, fill = "gray70") +
  labs(
    title = "Analisi dei Residui del Modello Random Forest",
    subtitle = "I residui mostrano l'errore di stima (Reale - Previsto) lungo la vita utile",
    x = "RUL Reale",
    y = "Residuo (Reale - Previsto)"
  ) +
  theme_minimal()
```


Analisi dei Residui del Modello Random Forest

I residui mostrano l'errore di stima (Reale – Previsto) lungo la vita utile



L'esame della distribuzione dei residui mostra che in corrispondenza di un RUL reale prossimo allo zero la curva di tendenza e la dispersione dei punti convergono esattamente sulla linea dello zero, ciò conferma che il modello è affidabile proprio quando la criticità è massima, fornendo stime precise quando il motore si trova a pochi cicli dalla rottura. Nelle zone centrali e a destra del grafico i residui risultano avere un'oscillazione più ampia, con una leggera tendenza verso la distorsione sistematica. Tale andamento riflette la natura non lineare del degrado iniziale e l'effetto di soglia, confermando quindi che la stima a lungo termine presenta un margine di incertezza fisiologico intrinseco al dataset.